

I've **reconstructed the patch** and verified it's on the sandbox. You can download it here:

 **Download: `apb_unified_pricing_patch.zip`**

If you still can't access the zip in your UI, I've also pasted **every file** from the patch below so you can copy/paste directly into your project.

What this patch does (quick)

- **Merges the two Step-3 tables into ONE editable table** that shows *deliverables and their components* together.
- Keeps header **Cadence** (replaces "Type") and makes cadence **editable** at either level.
- **Propagates cadence/months** from a deliverable to its components (unless a component is manually overridden).
- **"Re-build Scenario"** now acts as **Save** (server endpoint + local fallback).
- Adds a **Gantt bridge**: when the Gantt timeline changes (by user drag or AI), pricing **Hours/Months/Cost** update live.
- Introduces a lightweight **Working Scenario** store that carries the active scenario across steps.

Active scenario name: `Working Scenario` (id: `working`).

You can rename by calling:

```
ScenarioStore.setState({ id: "working", name: "Working Scenario v2"
});
```

How to install (5–10 min)

Mount API routes (FastAPI)

Add this to your server bootstrap:

```
from backend.scenario_api import router as scenario_router
```

```
app.include_router(scenario_router, prefix="/api/scenario",
tags=["scenario"])
```

1.

Replace the two pricing tables with ONE container

In your Step-3 page, remove/hide the two old tables and include this partial (or paste the snippet inline):

```
{% include "templates/partials/pricing-one-table.html" %}
```

2.

Hydrate the table from your existing selection (Step-2 output or your AI planner):

```
<script>
  APBOneTable.hydrateFrom({
    deliverables: [
      {
        id: "strategy_campaign_deck",
        title: "Creative Strategy / Campaign Plan Deck",
        dept: "Strategy",
        cadence: "One-Time",
        months: 0,
        hours: 220,
        rate: 195,
        components: [
          { id:"activation_ideas", title:"Activation Ideas", hours:40
},
          { id:"art_direction", title:"Art Direction", hours:29 },
          // ... all components preserved
        ]
      },
      // ...other deliverables
    ]
  });
</script>
```

3.

Wire the Gantt → Pricing sync

Wherever your Gantt changes a task:

```
GanttBridge.emitChange({
  deliverableId: "strategy_campaign_deck",
  start: "2025-11-01",
  end: "2026-01-15",
  durationDays: 50,
  resources: ["Designer", "PM"] // optional
});
```

4.

5. **Use “Re-build Scenario (Save)”**

This saves to `localStorage` and `POST /api/scenario/save`.

Files included in the zip (full source below)

```
static/js/scenario-store.js
static/js/pricing-one-table.js
static/js/gantt-bridge.js
static/css/pricing-one-table.css
templates/partials/pricing-one-table.html
backend/scenario_api.py
```

static/js/scenario-store.js

```
// scenario-store.js
// Central store: carries the “Working Scenario” across steps and
// keeps
// Step-3 Pricing and Step-4 Gantt in sync via pub/sub.

(function () {
  const _listeners = new Set();
  const DEFAULTS = { hoursPerDay: 6, currency: "USD", blendedRate: 195
};

  const ScenarioStore = {
    state: {
```

```

    id: "working",
    name: "Working Scenario",
    createdAt: new Date().toISOString(),
    updatedAt: new Date().toISOString(),
    currency: DEFAULTS.currency,
    blendedRate: DEFAULTS.blendedRate,
    hoursPerDay: DEFAULTS.hoursPerDay,
    deliverables: [],
    totals: { hours: 0, oneTimeCost: 0, monthlyHours: 0,
monthlyCost: 0, grandTotal12: 0 }
  },
  subscribe(fn) { _listeners.add(fn); return () =>
_listeners.delete(fn); },
  emit() { for (const fn of Array.from(_listeners)) try {
fn(this.state); } catch(_){} },
  setState(patch) {
    Object.assign(this.state, patch || {});
    this.recompute();
    this.state.updatedAt = new Date().toISOString();
    this.emit();
  },
  load(scenario) {
    if (!scenario) return;
    this.state = Object.assign({}, this.state, scenario);
    this.recompute();
    this.emit();
  },
  upsertDeliverable(d) {
    const idx = this.state.deliverables.findIndex(x => x.id ===
d.id);
    if (idx >= 0) this.state.deliverables[idx] = Object.assign({},
this.state.deliverables[idx], d);
    else this.state.deliverables.push(Object.assign({components:[],
tasks:[], resources:[]}, d));
    this.recompute(); this.emit();
  },
  updateDeliverable(did, patch) {
    const d = this.state.deliverables.find(x => x.id === did);

```

```

    if (!d) return;
    Object.assign(d, patch || {});
    if (patch && patch.cadence) {
      // propagate unless component has an explicit override
      for (const c of d.components || []) if (!c._customCadence) {
        c.cadence = d.cadence; c.months = d.months;
      }
    }
    this.recompute(); this.emit();
  },
  updateComponent(did, cid, patch) {
    const d = this.state.deliverables.find(x => x.id === did);
    if (!d) return;
    const c = (d.components || []).find(x => x.id === cid);
    if (!c) return;
    Object.assign(c, patch || {});
    if (patch && (patch.cadence || patch.months !== null))
c._customCadence = true;
    this.recompute(); this.emit();
  },
  upsertMany(deliverables) {
    for (const d of deliverables || []) this.upsertDeliverable(d);
    this.recompute(); this.emit();
  },
  recompute() {
    let hours = 0, oneTimeCost = 0, monthlyHours = 0, monthlyCost =
0;

    const rate = Number(this.state.blendedRate ||
DEFAULTS.blendedRate);
    for (const d of this.state.deliverables) {
      const dh = (typeof d.hours === "number") ? d.hours
: (d.components || []).reduce((s,c)=> s +
(Number(c.hours||0)||0), 0);
      d.hours = Math.round(dh*10)/10;
      d.rate = d.rate || rate;
      const cad = d.cadence || "One-Time";
      if (cad === "One-Time") {
        d.price = Math.round(d.hours * d.rate);

```

```

        oneTimeCost += d.price;
    } else {
        const months = Number(d.months || 1);
        d.price = Math.round(d.hours * d.rate * months);
        monthlyHours += d.hours;
        monthlyCost += Math.round(d.hours * d.rate);
    }
    hours += d.hours;
    for (const c of d.components || []) {
        c.rate = c.rate || rate;
        c.hours = Number(c.hours || 0);
        const ccad = c.cadence || d.cadence || "One-Time";
        if (ccad === "One-Time") c.price = Math.round(c.hours *
c.rate);
        else {
            const months = Number(c.months || d.months || 1);
            c.price = Math.round(c.hours * c.rate * months);
        }
    }
}
this.state.totals.hours = Math.round(hours*10)/10;
this.state.totals.oneTimeCost = oneTimeCost;
this.state.totals.monthlyHours = Math.round(monthlyHours*10)/10;
this.state.totals.monthlyCost = monthlyCost;
this.state.totals.grandTotal12 = oneTimeCost + (monthlyCost *
12);
},
async save() {
    try { localStorage.setItem("working_scenario",
JSON.stringify(this.state)); } catch(_){}
    try {
        const r = await fetch("/api/scenario/save", {
            method: "POST",
            headers: { "Content-Type": "application/json" },
            body: JSON.stringify(this.state)
        });
        if (!r.ok) throw new Error(await r.text());
    }
}

```

```

        } catch (e) { console.warn("Scenario save failed (local copy
saved):", e.message); }
        this.emit();
    },
    updateFromGantt(payload) {
        const { deliverableId, durationDays, resources } = payload ||
{};
        const d = this.state.deliverables.find(x => x.id ===
deliverableId);
        if (!d) return;
        const hrsPerDay = Number(this.state.hoursPerDay ||
DEFAULTS.hoursPerDay);
        const resCount = Number((Array.isArray(resources) ?
resources.length : (resources||1)) || 1);
        d.hours = Math.round(Math.max(1, Number(durationDays||0)) *
hrsPerDay * resCount * 10) / 10;

        if (d.cadence && d.cadence !== "One-Time" && payload.start &&
payload.end) {
            const s = new Date(payload.start), e = new Date(payload.end);
            const months = Math.max(1, (e.getFullYear() - s.getFullYear())
* 12 + (e.getMonth() - s.getMonth()) + 1);
            d.months = months;
            for (const c of d.components || []) if (!c._customCadence)
c.months = months;
        }
        this.recompute(); this.emit();
    }
};

window.ScenarioStore = ScenarioStore;
document.addEventListener("gantt:changed", (ev) =>
ScenarioStore.updateFromGantt(ev.detail || {}));

// bootstrap from localStorage if available
try {
    const saved = localStorage.getItem("working_scenario");
    if (saved) ScenarioStore.load(JSON.parse(saved));
}

```

```
    } catch(_) {}  
  })();
```

static/js/pricing-one-table.js

```
// pricing-one-table.js  
// Builds ONE unified editable Pricing table (deliverables +  
components).  
(function () {  
  const CADENCE_OPTIONS = ["One-Time", "Monthly", "Quarterly"];  
  
  function h(tag, attrs={}, ...children) {  
    const el = document.createElement(tag);  
    for (const [k,v] of Object.entries(attrs || {})) {  
      if (k === "class") el.className = v;  
      else if (k.startsWith("on") && typeof v === "function")  
el.addEventListener(k.substring(2).toLowerCase(), v);  
      else if (k === "dataset") Object.assign(el.dataset, v);  
      else if (v != null) el.setAttribute(k, v);  
    }  
    for (const c of children) el.appendChild(typeof c === "string" ?  
document.createTextNode(c) : c);  
    return el;  
  }  
  
  const money = (n) => (isFinite(n) ? `$$${Number(n).toLocaleString()}`  
: "-");  
  
  const numberInput = (value, onChange, attrs={}) => {  
    const input = h("input", Object.assign({ type:"number",  
value:String(value ?? ""), class:"apb-input apb-num" }, attrs));  
    input.addEventListener("change", e =>  
onChange(Number(e.target.value)));  
    input.addEventListener("blur", e =>  
onChange(Number(e.target.value)));  
    return input;  
  };
```



```

const cadenceSelect = (value, onChange) => {
  const sel = h("select", { class:"apb-input apb-sel" });
  for (const opt of CADENCE_OPTIONS) sel.appendChild(h("option", {
value:opt, selected:opt===value }, opt));
  sel.addEventListener("change", e => onChange(e.target.value));
  return sel;
};

function rowDeliverable(d) {
  const tr = h("tr", { class:"apb-row-deliv", dataset:{ id:d.id } });
  const toggle = h("button", { class:"apb-exp" }, "►");
  toggle.addEventListener("click", () => {
    const open = tr.classList.toggle("open");
    toggle.textContent = open ? "▼" : "►";
    for (const r of
document.querySelectorAll(`tr[data-parent='${d.id}']`))
r.style.display = open ? "" : "none";
  });

  tr.appendChild(h("td", { class:"apb-cell-title" }, toggle, " ",
h("strong", {}, d.title)));
  tr.appendChild(h("td", {}, cadenceSelect(d.cadence || "One-Time",
(val)=> ScenarioStore.updateDeliverable(d.id, { cadence: val }))) );
  tr.appendChild(h("td", {}, numberInput(d.months ||
(d.cadence==="Monthly"?12:(d.cadence==="Quarterly"?4:0)), (val)=>
ScenarioStore.updateDeliverable(d.id, { months: val }), { min:"0"
})));
  tr.appendChild(h("td", {}, numberInput(d.hours || 0, (val)=>
ScenarioStore.updateDeliverable(d.id, { hours: val }), { min:"0",
step:"0.1" })));
  tr.appendChild(h("td", {}, numberInput(d.rate ||
ScenarioStore.state.blendedRate || 195, (val)=>
ScenarioStore.updateDeliverable(d.id, { rate: val }), { min:"0",
step:"1" })));
  tr.appendChild(h("td", { class:"apb-right" }, money(d.price)));

```

```

        const resInput = h("input", { type:"text", class:"apb-input
apb-txt", value:(d.resources||[]).join(", ") });
        resInput.addEventListener("change", (e)=> {
            const list =
String(e.target.value||"").split(",").map(s=>s.trim()).filter(Boolean)
;
            ScenarioStore.updateDeliverable(d.id, { resources: list });
        });
        tr.appendChild(h("td", {}, resInput));

        const saveBtn = h("button", { class:"apb-btn apb-sm", onclick:
( )=> ScenarioStore.save() }, "Save");
        tr.appendChild(h("td", { class:"apb-right" }, saveBtn));
        return tr;
    }

    function rowComponent(d, c) {
        const tr = h("tr", { class:"apb-row-comp", dataset:{ id:c.id,
parent:d.id }});
        tr.appendChild(h("td", { class:"apb-cell-title" }, h("span", {
class:"apb-spacer" }, "└─ "), c.title));
        tr.appendChild(h("td", {}, cadenceSelect(c.cadence || d.cadence ||
"One-Time", (val)=> ScenarioStore.updateComponent(d.id, c.id, {
cadence: val }))) ));
        tr.appendChild(h("td", {}, numberInput(c.months || d.months || 0,
(val)=> ScenarioStore.updateComponent(d.id, c.id, { months: val }), {
min:"0" })))));
        tr.appendChild(h("td", {}, numberInput(c.hours || 0, (val)=>
ScenarioStore.updateComponent(d.id, c.id, { hours: val }), { min:"0",
step:"0.1" })))));
        tr.appendChild(h("td", {}, numberInput(c.rate || d.rate ||
ScenarioStore.state.blendedRate || 195, (val)=>
ScenarioStore.updateComponent(d.id, c.id, { rate: val }), { min:"0",
step:"1" })))));
        tr.appendChild(h("td", { class:"apb-right" }, money(c.price)));

        const tasksInput = h("input", { type:"text", class:"apb-input
apb-txt", value:(c.tasks||[]).join(", ") });

```

```

        tasksInput.addEventListener("change", (e)=> {
            const list =
String(e.target.value||"").split(",").map(s=>s.trim()).filter(Boolean)
;
            ScenarioStore.updateComponent(d.id, c.id, { tasks: list });
        });
        tr.appendChild(h("td", {}, tasksInput));
        tr.appendChild(h("td", { class:"apb-right" }, ""));
        return tr;
    }

```

```

function totalsRow(state) {
    const tot = state.totals || {};
    const tr = h("tr", { class:"apb-row-totals" });
    tr.appendChild(h("td", { colspan:"3" }, h("strong", {},
"Totals"))));
    tr.appendChild(h("td", {}, String(tot.hours || 0)));
    tr.appendChild(h("td", {}, ""));
    tr.appendChild(h("td", { class:"apb-right" },
money((tot.oneTimeCost||0) + (tot.monthlyCost||0))));
    tr.appendChild(h("td", {}, ""));
    const rebuild = h("button", { class:"apb-btn", onclick: ()=>
ScenarioStore.save() }, "Re-build Scenario (Save)");
    tr.appendChild(h("td", { class:"apb-right" }, rebuild));
    return tr;
}

```

```

function render(tableId="apb-one-table") {
    const mount = document.getElementById(tableId);
    if (!mount) return;
    mount.innerHTML = "";

    const tbl = h("table", { class:"apb-table" });
    tbl.appendChild(
        h("thead", {},
            h("tr", {},
                h("th", {}, "Deliverable / Component"),
                h("th", {}, "Cadence"),

```

```

        h("th", {}, "Months"),
        h("th", {}, "Hours"),
        h("th", {}, "Rate"),
        h("th", {}, "Cost"),
        h("th", {}, "Tasks / Resources"),
        h("th", {}, "Actions")
    )
)
);
const tb = h("tbody");
const state = ScenarioStore.state;

for (const d of state.deliverables) {
    tb.appendChild(rowDeliverable(d));
    for (const c of (d.components || [])) {
        const r = rowComponent(d, c);
        r.style.display = "none"; // shown when user expands
deliverable
        tb.appendChild(r);
    }
}
tb.appendChild(totalsRow(state));
tbl.appendChild(tb);
mount.appendChild(tbl);
}

document.addEventListener("DOMContentLoaded", ()=> render());
ScenarioStore.subscribe(()=> render());

// Public helper to hydrate from your existing selection object
window.APBOneTable = {
    mount: render,
    hydrateFrom(raw) {
        if (!raw) return;
        ScenarioStore.upsertMany(raw.deliverables || []);
    }
};
})();

```

static/js/gantt-bridge.js

```
// gantt-bridge.js
// The Gantt calls this to push timeline edits into the Scenario
store.
(function(){
    function emitChange(payload) {
        document.dispatchEvent(new CustomEvent("gantt:changed", { detail:
payload || {} }));
    }
    window.GanttBridge = { emitChange };
})();
```

static/css/pricing-one-table.css

```
.apb-table { width: 100%; border-collapse: collapse; font-size:
0.95rem; }
.apb-table th, .apb-table td { border-bottom: 1px solid #2f3440;
padding: 8px 10px; }
.apb-row-deliv { background: #1e2430; }
.apb-row-comp { background: #151a24; }
.apb-row-totals { background: #232a36; }
.apb-cell-title { width: 32%; white-space: nowrap; }
.apb-right { text-align: right; }
.apb-input { width: 100%; box-sizing: border-box; background: #0f131b;
color: #e6eefc; border: 1px solid #2b3241; border-radius: 4px;
padding: 6px 8px; }
.apb-input:focus { outline: none; border-color: #6aa0ff; }
.apb-num { text-align: right; }
.apb-sel { min-width: 120px; }
.apb-txt { min-width: 180px; }
.apb-exp { background: none; color: #a9b6d8; border: none; font-size:
14px; cursor: pointer; }
.apb-btn { cursor: pointer; padding: 6px 10px; border-radius: 6px;
border: 1px solid #3a4154; background: #2b3241; color: #e6eefc; }
.apb-btn.apb-sm { padding: 4px 8px; font-size: 12px; }
```

```
.apb-spacer { color: #7684a5; margin-right: 4px; }
```

templates/partial/pricing-one-table.html

```
<section id="pricing-unified">
  <h3> Pricing – Unified</h3>
  <div id="apb-one-table"></div>
</section>

<link rel="stylesheet" href="/static/css/pricing-one-table.css">
<script src="/static/js/scenario-store.js"></script>
<script src="/static/js/pricing-one-table.js"></script>
<script src="/static/js/gantt-bridge.js"></script>
```

backend/scenario_api.py

```
from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import Any, Dict
import time

router = APIRouter()

class Scenario(BaseModel):
    id: str
    name: str
    createdAt: str | None = None
    updatedAt: str | None = None
    currency: str | None = "USD"
    blendedRate: float | None = None
    hoursPerDay: float | None = None
    deliverables: list[dict]
    totals: dict | None = None

# Simple in-memory persistence; swap with your DB if needed.
_INMEM: Dict[str, Dict[str, Any]] = {}
```

```
@router.post("/save")
async def save_scenario(s: Scenario):
    data = s.dict()
    data["_serverSavedAt"] = time.time()
    _INMEM[data["id"]] = data
    return {"ok": True, "id": data["id"], "serverSavedAt":
data["_serverSavedAt"]}

@router.get("/active")
async def get_active():
    if "working" in _INMEM:
        return {"ok": True, "scenario": _INMEM["working"]}
    raise HTTPException(status_code=404, detail="No active scenario
found")
```

Final notes

- **Cadence** is the single source of truth ("Type" is removed). Editing cadence at the deliverable level will **auto-update** its components unless a component has been manually overridden.
- **Re-build Scenario = Save** (local + server).
- Gantt changes → fire `GanttBridge.emitChange(...)` → pricing updates instantly.
- **Working Scenario** is the active data object carried across steps.

If you still can't pull the zip from the link, you can copy each file above verbatim into your Replit project and you're good to go.